Technical Task: Path Extrapolation

- Build an interactive canvas with draggable points in Flutter.
- Generate a smooth continuous path that passes through all points in order.
- Place equal-size circles along the path under strict geometric constraints.
- Clip the final circle exactly at the path end boundary.

Core Requirements

1. **Reduction Step**
    - Start with a collection of candidate points on one side (about 3 points).
    - Project each candidate point to a reference (such as a line or axis) derived to best fit these candidate points.
    - Among the projected instances, deterministically select the one that is at the max distance from P2.
    - This derived point must be used as one of the path points.
2. **Build the 5-Point Path**
    - In addition to the reduced point above, use 4 other draggable points.
    - Total path input must be exactly 5 points (`1 reduced + 4 other`).
    - Generate a smooth continuous path that passes through these 5 points in order.
3. **Circle Packing Along Path**
    - Place fixed-diameter circles along the path.
    - Circles must be laid edge-to-edge with no gap.
    - The first circle must be positioned so the **circle boundary** starts exactly at the path start point.
    - Must work across arbitrary path shapes and curvature.
4. **Exact End Clipping**
    - Any portion of the final circle beyond the path end must be clipped precisely at the endpoint boundary.
    - No overshoot, visible gap, or unstable flicker at the tail.
5. **Projection Visualization**
    - Render debug overlays for the projection/reduction step (candidate points, reference/projection guides, selected reduced point).
    - Selection must be deterministic for the same input.

Visual Requirements

- Interactive canvas showing all draggable points at all times.
- Clear visual distinction between:
  - candidate points used in reduction,
  - the selected reduced point,
  - the 4 additional path points.
- Path should remain clearly visible against background during interaction.
- Circle packing should be visually continuous (no visible spacing between adjacent circles).
- Endpoint clipping should be visibly exact at the last path point.

Bonus

- Animate circle placement/progression along the path while preserving the same geometric constraints.
- Provide toggles (or equivalent controls) to show/hide:
  - projection/reduction debug overlays,
  - circle packing layer,
  - clipping boundary indicator.

Engineering Constraints

- Keep geometry logic separate from painter/render code.
- No hidden mutation inside render callbacks.
- Deterministic output for identical inputs.
- Handle edge cases (straight, bent, vertical, horizontal, inclined paths).

Deliverables

- Source code.
- `README.md` with setup, design decisions, and edge cases.
- Demo(s) illustrating the tool handling various edge cases, including:
  - Bent/curved path
  - Straight path
  - Vertical path
  - Any other degenerate or challenging configurations

Reference Figure

Projection / Reduction	Build 5-Point Path	Edge-to-Edge Circle Packing	Exact End Clipping